Higher-Order Functions & Functional Patterns

11 min read · Updated 17 Jul 2026 · Topic 11 of 23

Bookmark · Completed

In JavaScript, functions are **values** — you can store them in variables, pass them as arguments, and return them from other functions. A **higher-order function (HOF)** is any function that does one of the last two: takes a function as an argument, returns a function, or both. This single idea underpins `map` / `filter` / `reduce`, event handlers, `Promise.then`, middleware, currying, and composition — which is why interviewers lean on it so heavily. This lesson builds the concept from first-class functions up to composition and point-free style, and re-implements the array iterators from scratch (a near-universal interview task).

First-class functions & callbacks

Saying functions are **first-class** means they are ordinary values with no special status: assignable, passable, returnable, storable in arrays and objects. A **callback** is simply a function you hand to another function to be called later — synchronously (`map`) or asynchronously (`setTimeout`, `.then`).

JS Copy

```
const twice = (f, x) => f(f(x)); // f is a callback, twice is a HOF
console.log(twice((n) => n + 3, 10)); // => 16
```

A HOF that **returns** a function lets you capture configuration in a closure and produce a specialized function — the basis of memoization, throttling, and `once`.

JS Copy

```
  let called = false, result;
  return function (...args) {
    if (!called) { called = true; result = fn.apply(this, args); }
    return result;
  };
}
const init = once(() => { console.log("init"); return 42; });
console.log(init(), init()); // logs "init" once, then => 42 42
```

`map`, `filter`, `reduce` — and how they actually work

These three are the workhorses of functional JS. All are **non-mutating**: they read the source array and return a new value, leaving the original untouched. `map` transforms each element 1:1, `filter` keeps a subset, and `reduce` folds the whole array into a single accumulated value (which can itself be an array or object).

Method	Callback returns	Result	Length
`map`	the transformed element	new array	same as source
`filter`	a boolean (keep?)	new array	≤ source
`reduce`	the next accumulator	single value	n/a

Implementing these from scratch is the classic interview question, so know them cold. The details that separate a strong answer from a weak one: the callback receives `(element, index, array)`, `filter`/`map` skip holes in sparse arrays, and `reduce` has special behavior when no initial value is given.

`map` polyfill

JS Copy

```
Array.prototype.myMap = function (fn, thisArg) {
  const out = [];
  for (let i = 0; i < this.length; i++) {
```

```
  if (i in this) out[i] = fn.call(thisArg, this[i], i, this);
  return out;
};
console.log([1, 2, 3].myMap((x) => x * 2)); // => [ 2, 4, 6 ]
```

`filter` polyfill

JS Copy

```
Array.prototype.myFilter = function (fn, thisArg) {
  const out = [];
  for (let i = 0; i < this.length; i++) {
    if (i in this && fn.call(thisArg, this[i], i, this)) out.push(this[i])
  }
  return out;
};
console.log([1, 2, 3, 4].myFilter((x) => x % 2 === 0)); // => [ 2, 4 ]
```

`reduce` polyfill

The subtle part is the missing-initial-value case: `reduce` then uses the **first element** as the seed and starts iterating from the second, and it **throws** on an empty array with no seed.

JS Copy

```
Array.prototype.myReduce = function (fn, initial) {
  let acc, start;
  if (arguments.length >= 2) { acc = initial; start = 0; }
  else {
    let i = 0;
    while (i < this.length && !(i in this)) i++;
    if (i >= this.length) throw new TypeError("Reduce of empty array with
    acc = this[i]; start = i + 1;
  }
  for (let i = start; i < this.length; i++) {
    if (i in this) acc = fn(acc, this[i], i, this);
  }
  return acc;
```

`reduce` is the general one — the others are special cases

`reduce` can express `map` and `filter`, plus aggregations they can't. Interviewers love asking you to build `map`/`filter` *with* `reduce` to check you understand the fold.

JS Copy

```
// sum
console.log([1, 2, 3, 4].reduce((a, b) => a + b, 0)); // => 10

// flatten one level
console.log([[1, 2], [3, 4], [5]].reduce((acc, cur) => acc.concat(cur), []

// group-by
const people = [
  { name: "Ann", dept: "eng" },
  { name: "Bob", dept: "sales" },
  { name: "Cy", dept: "eng" },
];
const byDept = people.reduce((acc, p) => {
  (acc[p.dept] ||= []).push(p.name);
  return acc;
}, {});
console.log(byDept); // => { eng: [ 'Ann', 'Cy' ], sales: [ 'Bob' ] }

// map and filter, expressed as reduce
const doubled = [1, 2, 3].reduce((acc, x) => { acc.push(x * 2); return acc
const evens = [1, 2, 3, 4].reduce((acc, x) => { if (x % 2 === 0) acc.push(
console.log(doubled, evens); // => [ 2, 4, 6 ] [ 2, 4 ]
```

Pure functions, side effects & immutability

A **pure function** satisfies two rules: (1) its return value depends only on its arguments — same input always gives same output — and (2) it causes **no side effects** (no mutating external state, no I/O, no DOM writes, no `console.log`, no touching

change to the world outside the function's return value.

The most common accidental impurity is **mutating an argument**. The impure version below changes its caller's array; the pure version returns a fresh one.

JS Copy

```
// impure: mutates the input array
function addItemImpure(cart, item) { cart.push(item); return cart; }
const c1 = ["a"];
const r1 = addItemImpure(c1, "b");
console.log(c1, c1 === r1); // => [ 'a', 'b' ] true  (original was mutated

// pure: builds a new array, leaves input alone
function addItemPure(cart, item) { return [...cart, item]; }
const c2 = ["a"];
const r2 = addItemPure(c2, "b");
console.log(c2, r2, c2 === r2); // => [ 'a' ] [ 'a', 'b' ] false
```

Immutability is the discipline of never mutating existing data — you produce new values with spread (`...`), `map`, `filter`, or `slice` instead of `push`/`splice`/direct assignment. This is *why* React state updates use `{ ...state }` and `[...arr]`: change detection relies on reference equality (`prev !== next`), which mutation defeats. Note that spread is a **shallow** copy — nested objects you don't touch are shared by reference (a feature, not a bug: it makes updates cheap), so you must spread every level you intend to change.

JS Copy

```
const state = { user: { name: "Ann", age: 30 }, tags: ["x"] };
const next = { ...state, user: { ...state.user, age: 31 } };
console.log(state.user.age, next.user.age); // => 30 31 (original untouche
console.log(state.user === next.user);      // => false (changed branch is
console.log(state.tags === next.tags);      // => true  (untouched branch
```

Function composition: `compose` and `pipe`

Composition chains single-argument functions so the output of one feeds the input of the next. `compose` runs **right-to-left** (mathematical order, `f(g(x))`); `pipe` runs **left-to-right** (reading order). Both are tiny — and they *are* `reduce` / `reduceRight` over the function list, which ties this section back to folds.

JS Copy

```
const compose = (...fns) => (x) => fns.reduceRight((acc, fn) => fn(acc), x
const pipe    = (...fns) => (x) => fns.reduce((acc, fn) => fn(acc), x);

const inc = (n) => n + 1;
const dbl = (n) => n * 2;

console.log(compose(inc, dbl)(5)); // inc(dbl(5)) => 11
console.log(pipe(inc, dbl)(5));    // dbl(inc(5)) => 12
```

Composition lets you name each small step once and assemble pipelines declaratively — `const clean = pipe(trim, toLower, dedupe)` reads like a spec.

Point-free style — and when functional hurts

Point-free (tacit) style means defining a function without naming its argument, by passing an existing function directly instead of wrapping it in an arrow.

JS Copy

```
const words = ["1", "22", "333"];
console.log(words.map((s) => s.length)); // pointful
const toLen = (s) => s.length;
console.log(words.map(toLen));           // point-free => [ 1, 2, 3 ]
```

Functional style **helps** when it removes noise: a named `pipe` of small pure functions, or `list.filter(isActive).map(toName)`, is more readable and testable than an imperative loop with a temp array. It **hurts** when it obscures: over-eager point-free chains, deeply nested `compose`, or clever currying can produce code that's shorter but

`["1","2"].map(parseInt)` breaks because `map` passes `(value, index)` and `parseInt`'s second arg is the radix — so favor clarity, and don't go point-free just to prove you can.

Common interview questions

- **"Implement `map` / `filter` / `reduce` from scratch."** Loop with a plain `for`, call the callback with `(element, index, array)`, and for `reduce` handle the no-initial-value case (seed with the first element, start at index 1, throw `TypeError` on an empty array with no seed). Return a new array for `map` / `filter`; return the accumulator for `reduce`. See the polyfills above.
- **"What is a pure function?"** One whose output depends only on its inputs and which produces no side effects — no mutation of external state, no I/O, no reliance on `Date` / `Math.random`. Given the same arguments it always returns the same result, which makes it testable, cacheable, and safe to reorder.
- **"What's a higher-order function? Give examples."** A function that takes and/or returns a function. Built-ins: `map`, `filter`, `reduce`, `sort`, `addEventListener`, `setTimeout`, `Promise.then`. Returning-a-function examples: `once`, a debounce factory, a curried adder.
- **"Difference between `map` and `forEach`?"** `map` returns a new array of the callback's return values and is meant for transformation; `forEach` returns `undefined` and is meant for side effects. Use `map` only when you use the result — mapping purely for a side effect is a smell.
- **"Implement `compose` / `pipe`."** `pipe = (...fns) ⇒ x ⇒ fns.reduce((acc, fn) ⇒ fn(acc), x)`; `compose` is the same with `reduceRight`. Explain that `compose` is right-to-left and `pipe` is left-to-right.
- **"Why does React need immutable updates?"** State-change detection compares references (`prev ≠= next`). Mutating in place keeps the same reference, so the update is missed. Spreading a new object/array gives a new reference and a correct re-render.

Gotchas / traps

empty.

JS Copy

```
try { [].reduce((a, b) => a + b); }
catch (e) { console.log(e.name + ": " + e.message); }
// => TypeError: Reduce of empty array with no initial value
```

Without an initial value, the callback is skipped for a single-element array. The lone element becomes the accumulator and `reduce` returns it un-transformed — so any per-element logic (parsing, coercion) never runs.

JS Copy

```
let calls = 0;
const only = [42].reduce((a, b) => { calls++; return a + b; });
console.log(only, "calls=" + calls); // => 42 calls=0
```

Reducing without a numeric seed can silently string-concatenate. If the elements are strings, `a + b` concatenates; if you forget the `0` seed on numbers-as-strings you get `"123"`, not `6`. Seed explicitly and with the right type.

JS Copy

```
console.log(["1", "2", "3"].reduce((a, b) => a + b));    // => 123  (strin
console.log(["1", "2", "3"].reduce((a, b) => a + b, 0)); // => 0123 (strin
console.log([1, 2, 3].reduce((a, b) => a + b, 0));       // => 6
```

Mutating objects inside `map` defeats the purpose. `map` gives you a new *array*, but if the callback mutates the shared object and returns it, both arrays' elements point at the same mutated objects — you've created a fresh array of stale references. Spread each object to stay immutable.

JS Copy

```
console.log(objs[0].n, bad[0] === objs[0]); // ⇒ 10 true  (shared!)
const good = [{ n: 1 }, { n: 2 }].map((o) ⇒ ({ ...o, n: o.n * 10 }));
console.log(good[0].n, good[0] === objs[0]); // ⇒ 100 false (fresh object
```

`array.map(parseInt)` **is the classic point-free trap.** `map` passes `(value, index, array)`, and `parseInt(string, radix)` reads that index as a radix: `["1","2","3"].map(parseInt)` yields `[1, NaN, NaN]` (parse "2" in base 1 → NaN). Wrap it: `.map((s) ⇒ parseInt(s, 10))` or `.map(Number)`.

QUICK CHECK — no marks, just for you.

1. What does this log?

JAVASCRIPT Copy

```
console.log(["1", "2", "3"].map(parseInt));
```

- ◯ `[ 1, 2, 3 ]`
- ◯ `[ '1', '2', '3' ]`
- ◯ `[ 1, NaN, NaN ]`
- ◯ `[ NaN, NaN, NaN ]`

Check answer

2. What does this log?

JAVASCRIPT Copy

```
const compose = (...fns) => (x) => fns.reduceRight((acc, fn) => fn(acc), x);
const inc = (n) => n + 1;
```

- ○ 11
- ○ 12
- ○ 16
- ○ 10

Check answer

3. What does this log?

JAVASCRIPT Copy

```
console.log(["1", "2", "3"].reduce((a, b) => a + b, 0));
```

- ○ 6
- ○ '6'
- ○ 123
- ○ 0123

Check answer

JavaScript Interview Kit

LEARN MORE

ARTICLES & DOCS

First-class functions (MDN)
MDN

VIDEOS



Higher-order functions, simply
YouTube



HOF simplified
YouTube



Higher-order functions (Coding Train)
YouTube

Hand-picked references for this lesson.